

Python 3 零基础入门教程

整理自菜鸟教程 runoob.com 及其他公开资源 | 2026-06-04

Python 3 零基础入门教程

> 来源：菜鸟教程 (runoob.com) 及其他公开资源整理 > 最后更新：2026-06-04

目录

1. Python 简介与环境安装 2. 第一个程序：Hello World 3. 基础语法（缩进、注释、标识符） 4. 变量与数据类型 5. 数字(Number) 6. 字符串(String) 7. 列表(List) 8. 元组(Tuple) 9. 集合(Set) 10. 字典(Dictionary) 11. 条件判断(if/else) 12. 循环(for/while) 13. 函数 14. 模块与包 15. 文件读写 16. 错误与异常处理 17. 面向对象编程 18. 推荐学习路线

1. Python 简介与环境安装

什么是 Python

Python 是一种解释型、面向对象、动态数据类型的高级程序设计语言。由 Guido van Rossum 于 1989 年底发明，第一个公开发行人于 1991 年。

特点：简单易学、可读性强、库丰富、跨平台（Windows/Linux/Mac）。

Python 3 vs Python 2

- 本教程基于 Python 3（Python 2 已于 2020 年停止维护） - Python 3 和 Python 2 不兼容，选择 Python 3

安装 Python

Windows：1. 前往 python.org 下载最新版安装包 2. 运行安装程序，勾选「Add Python to PATH」 3. 点击「Install Now」

Linux（Ubuntu/Debian）：

```
sudo apt update
sudo apt install python3 python3-pip
```

Mac：

```
brew install python3
```

验证安装：

```
python3 --version
# 或
python3 -V
```

查看 Python 版本

```
python -V
# 输出示例: Python 3.12.0
```

2. 第一个程序：Hello World

创建文件 hello.py：

```
#!/usr/bin/python3

print("Hello, World!")
```

运行：

```
python3 hello.py
```

输出：

```
Hello, World!
```

说明： - print() 是 Python 的内置函数，用于打印输出 - .py 是 Python 源码文件的后缀名 - Python 可以使用交互式模式（在终端输入 python3），也可以脚本模式（python3 xxx.py）

3. 基础语法

编码

Python 3 默认以 UTF-8 编码，所有字符串都是 Unicode。

```
# -*- coding: utf-8 -*-
```

标识符（变量名）

规则： - 第一个字符必须是字母或下划线 _ - 其他部分由字母、数字、下划线组成 - 大小写敏感：age 和 Age 是不同的 - 不能使用 Python 关键字

合法标识符：

```
age = 25          # 普通变量名
user_name = "Alice" # 下划线连接（推荐）
MAX_SIZE = 1024    # 全大写表示常量
calculate_area()    # 函数名，动词+名词
StudentInfo        # 类名，首字母大写（驼峰）
```

非法标识符：

```
2nd_place = "silver" # 错误：数字开头
user-name = "Bob"     # 错误：包含连字符
class = "Math"        # 错误：关键字
$price = 9.99         # 错误：特殊字符
```

Python 保留关键字

```
False, None, True, and, as, assert, async, await, break,
class, continue, def, del, elif, else, except, finally,
for, from, global, if, import, in, is, lambda, nonlocal,
not, or, pass, raise, return, try, while, with, yield
```

注释

```
# 这是单行注释

"""
这是多行注释
第二行
"""

"""
这也是多行注释
"""
```

缩进（最重要！）

Python 用缩进来表示代码块，不使用大括号 {}。同一个代码块的缩进空格数必须一致。

```
if True:
    print("True")  # 缩进 4 个空格
else:
    print("False")
```

错误示例（缩进不一致）：

```
if True:
    print("Answer")
    print("True")  # 缩进不一致，报错！
```

多行语句

```
# 使用反斜杠 \ 换行
total = item_one + \
    item_two + \
    item_three

# 在 [], {}, () 中不需要 \
total = ['item_one', 'item_two', 'item_three',
        'item_four', 'item_five']
```

多语句同行

```
import sys; x = 'runoob'; sys.stdout.write(x + '\n')
# 用分号；分隔，但不推荐这种写法
```

print 输出

```
print("Hello")
print("World")

# 不换行输出
print("Hello", end=" ")
print("World")
# 输出: Hello World
```

4. 变量与数据类型

变量赋值

Python 变量不需要声明类型，赋值即创建。

```
counter = 100    # 整型
miles = 1000.0   # 浮点型
name = "runoob"  # 字符串
is_active = True # 布尔值

# 多个变量同时赋值
a = b = c = 1    # 三个变量都指向同一个值
a, b, c = 1, 2, "hello" # 分别赋值
```

查看类型

```
print(type(10))    # <class 'int'>
print(type(3.14))  # <class 'float'>
print(type("hello")) # <class 'str'>
```

```
print(type(True))    # <class 'bool'>
```

Python 的 7 种标准数据类型

删除变量

```
del var
del var_a, var_b
```

5. 数字(Number)

三种数值类型

数值运算

```
5 + 4    # 加法 → 9
4.3 - 2  # 减法 → 2.3
3 * 7    # 乘法 → 21
2 / 4    # 除法（浮点数）→ 0.5
2 // 4   # 整除（向下取整）→ 0
17 % 3   # 取余 → 2
2 ** 5   # 乘方（2的5次方）→ 32
```

类型转换

```
int(3.14)    # → 3
float(10)    # → 10.0
complex(3, 4) # → 3+4j
```

数学函数

```
import math

abs(-10)     # 绝对值 → 10
math.ceil(4.1) # 向上取整 → 5
math.floor(4.9) # 向下取整 → 4
math.sqrt(25)  # 平方根 → 5.0
math.exp(1)    # e的指数 → 2.718...
max(1, 2, 3)   # 最大值 → 3
min(1, 2, 3)   # 最小值 → 1
round(3.14159, 2) # 四舍五入到2位 → 3.14
pow(2, 3)      # 幂运算 → 8
```

随机数

```
import random

random.random()      # [0, 1) 随机浮点数
random.randint(1, 10) # 1到10的随机整数
random.choice([1, 2, 3]) # 从列表中随机选取
random.shuffle([1, 2, 3, 4]) # 打乱列表
```

6. 字符串(String)

创建字符串

```
s1 = 'hello'    # 单引号
s2 = "world"    # 双引号
```

```
s3 = ""多行
字符串""      # 三引号（多行）
s4 = """"也可以
这样""""      # 三双引号
```

字符串索引和切片

```
s = '123456789'

print(s)      # 123456789
print(s[0])   # 1（第一个字符，索引从0开始）
print(s[-1])  # 9（最后一个字符）
print(s[0:-1]) # 12345678（从头到倒数第二个）
print(s[2:5])  # 345（索引2,3,4，不含5）
print(s[2:])   # 3456789（从索引2到末尾）
print(s[1:5:2]) # 24（步长2）
print(s * 2)   # 123456789123456789（重复）
print(s + '你好') # 123456789你好（拼接）
```

转义字符

```
print('hello\nworld') # 换行
print('hello\tworld') # Tab制表
print(r'hello\nworld') # r表示原始字符串，\n不会转义
# 输出: hello\nworld
```

字符串格式化

% 格式化（老式）：

```
print("我叫 %s 今年 %d 岁!" % ('小明', 10))
# 输出: 我叫 小明 今年 10 岁!
```

f-string（Python 3.6+，推荐）：

```
name = '小明'
age = 10
print(f"我叫 {name} 今年 {age} 岁!")
# 输出: 我叫 小明 今年 10 岁!

print(f"{1 + 2}") # 可以用表达式 → 3
```

常用字符串方法

```
s = "hello world"

s.upper()      # 'HELLO WORLD'（全部大写）
s.lower()      # 'hello world'（全部小写）
s.capitalize() # 'Hello world'（首字母大写）
s.title()      # 'Hello World'（每个单词首字母大写）
s.strip()      # 去掉首尾空格
s.replace("hello", "hi") # 'hi world'（替换）
s.split(" ")   # ['hello', 'world']（分割成列表）
"".join(['a', 'b']) # 'a b'（用空格连接列表）
len(s)         # 11（字符串长度）
s.find("world") # 6（查找子串，返回索引，找不到返回-1）
s.count("l")    # 3（统计字符出现次数）
s.startswith("he") # True（是否以某字符串开头）
s.endswith("ld")  # True（是否以某字符串结尾）
"123".isdigit() # True（是否全是数字）
```

7. 列表(List)

列表是 Python 中使用最频繁的数据类型。可变、有序、可重复。

创建列表

```
my_list = ['abcd', 786, 2.23, 'runoob', 70.2]
empty_list = []
```

列表索引和切片

```
print(my_list[0])    # 'abcd'
print(my_list[1:3])  # [786, 2.23] (索引1,2)
print(my_list[2:])   # [2.23, 'runoob', 70.2]
print(my_list[-1])   # 70.2 (最后一个)
```

列表操作

```
a = [1, 2, 3]
b = [4, 5, 6]

a + b      # [1, 2, 3, 4, 5, 6] (拼接)
a * 2      # [1, 2, 3, 1, 2, 3] (重复)
3 in a     # True (成员检查)

# 修改元素 (列表是可变的!)
a[0] = 9   # → [9, 2, 3]
a[1:3] = [8, 7] # → [9, 8, 7]
a[1:3] = [] # → [9] (删除指定范围的元素)
```

常用列表方法

```
nums = [3, 1, 2]

nums.append(4)    # [3, 1, 2, 4] (末尾添加)
nums.insert(1, 99) # [3, 99, 1, 2, 4] (指定位置插入)
nums.remove(1)    # [3, 99, 2, 4] (删除指定值)
nums.pop()        # 返回并删除最后一个元素 → 4
nums.pop(0)       # 返回并删除第一个元素 → 3
nums.sort()       # 排序 [1, 2, 3] (原地排序)
nums.reverse()    # 反转
len(nums)         # 长度
min(nums)         # 最小值
max(nums)         # 最大值
nums.count(2)     # 统计元素出现次数
nums.index(2)     # 查找索引
nums.clear()      # 清空列表
```

列表推导式

```
[x * 2 for x in range(5)]    # [0, 2, 4, 6, 8]
[x for x in range(10) if x % 2 == 0] # [0, 2, 4, 6, 8] (过滤)
```

8. 元组(Tuple)

元组与列表类似，但元素不可修改。用小括号 ()。

```
my_tuple = ('abcd', 786, 2.23, 'runoob', 70.2)
single_tuple = (20,) # 单个元素必须加逗号!
empty_tuple = ()
```

```
# 访问
print(my_tuple[0]) # 'abcd'
print(my_tuple[1:3]) # (786, 2.23)

# 元组不可修改！
# my_tuple[0] = 999 # 报错 TypeError

# 但元组里面可以包含可变对象
t = (1, [2, 3], 4)
t[1].append(5) # 可以 → (1, [2, 3, 5], 4)
```

9. 集合(Set)

无序、可变、元素唯一（去重）。用大括号 {} 或 set()。

空集合只能用 set(), {} 创建的是空字典！

```
s = {1, 2, 3, 3}
print(s) # {1, 2, 3} (自动去重)

# 集合运算
a = {1, 2, 3, 4}
b = {3, 4, 5, 6}

a - b    # {1, 2} (差集：在a但不在b)
a | b    # {1, 2, 3, 4, 5, 6} (并集)
a & b    # {3, 4} (交集)
a ^ b    # {1, 2, 5, 6} (对称差集)
1 in a   # True (成员测试)
```

10. 字典(Dictionary)

键值对(key-value)映射，可变。Python 3.7+ 保持插入顺序。

```
d = {'name': 'runoob', 'code': 1, 'site': 'www.runoob.com'}

# 访问
print(d['name'])    # 'runoob'
print(d.get('name')) # 'runoob' (推荐，键不存在返回None)
print(d.get('age', 18)) # 18 (键不存在返回默认值)

# 修改/添加
d['age'] = 10        # 添加新键值对
d['name'] = 'xiaoming' # 修改已有键的值

# 删除
del d['code']        # 删除键值对
d.pop('site')        # 删除并返回值

# 遍历
for key in d:        # 遍历键
    print(key, d[key])

for key, value in d.items(): # 遍历键值对
    print(key, value)

print(d.keys())      # dict_keys(['name', ...])
```

```
print(d.values())    # dict_values(['runoob', ...])
```

11. 条件判断 (if/else)

```
# 基本结构
if 条件:
    执行语句
elif 其他条件:
    执行语句
else:
    执行语句

# 示例
age = 18

if age < 18:
    print("未成年")
elif age < 60:
    print("成年人")
else:
    print("老年人")

# 简写（三元表达式）
result = "成年" if age >= 18 else "未成年"
```

比较运算符

```
== 等于
!= 不等于
> 大于
< 小于
>= 大于等于
<= 小于等于
```

逻辑运算符

```
and # 与：两边都为 True 才返回 True
or  # 或：任一边为 True 就返回 True
not # 非：取反
```

```
x = 5
print(x > 0 and x < 10) # True
print(x < 0 or x > 3)  # True
print(not x > 10)      # True
```

12. 循环

for 循环

```
# 遍历列表
fruits = ["苹果", "香蕉", "橙子"]
for fruit in fruits:
    print(fruit)

# range() 生成数字序列
for i in range(5):    # 0, 1, 2, 3, 4
```

```
print(i)

for i in range(2, 6):    # 2, 3, 4, 5
    print(i)

for i in range(0, 10, 2): # 0, 2, 4, 6, 8 ( 步长2 )
    print(i)

# 遍历字典
d = {'a': 1, 'b': 2}
for k, v in d.items():
    print(f'{k}: {v}')
```

while 循环

```
count = 0
while count < 5:
    print(count)
    count += 1
```

break 和 continue

```
# break: 跳出整个循环
for i in range(10):
    if i == 5:
        break    # 到5就跳出
    print(i)    # 输出 0,1,2,3,4

# continue: 跳过本次循环, 继续下一次
for i in range(10):
    if i == 5:
        continue    # 跳过5
    print(i)    # 输出 0,1,2,3,4,6,7,8,9
```

13. 函数

定义和调用

```
def 函数名(参数):
    """文档字符串 (说明函数做什么用) """
    函数体
    return 返回值

# 示例
def greet(name):
    """向某人打招呼"""
    return f"你好, {name} !"

print(greet("小明")) # 你好, 小明 !
```

参数类型

```
# 1. 位置参数 (必须按顺序传)
def add(a, b):
    return a + b

# 2. 默认参数
def greet(name, greeting="你好"):
    return f"{greeting}, {name} !" 
```

```
print(greet("小明"))      # 你好，小明！
print(greet("小明", "早上好")) # 早上好，小明！

# 3. 关键字参数（指定参数名，不按顺序）
print(greet(greeting="Hi", name="Tom"))

# 4. 可变参数 *args（任意数量参数，打包成元组）
def sum_all(*args):
    return sum(args)

print(sum_all(1, 2, 3, 4)) # 10

# 5. 关键字可变参数 **kwargs（打包成字典）
def show_info(**kwargs):
    for k, v in kwargs.items():
        print(f'{k}: {v}')

show_info(name="小明", age=18, city="北京")
```

Lambda 函数（匿名函数）

```
# 简单的一行函数
square = lambda x: x * x
print(square(5)) # 25

# 常用在 sort/sorted/map/filter 中
pairs = [(1, 'b'), (2, 'a')]
pairs.sort(key=lambda x: x[1]) # 按第二个元素排序
```

14. 模块与包

import 导入

```
import math      # 导入整个模块
print(math.sqrt(25)) # 5.0

from math import sqrt # 只导入某个函数
print(sqrt(25))      # 5.0

from math import sqrt, pi # 导入多个
print(pi)               # 3.14159...

from math import *     # 导入全部（不推荐）

import math as m       # 起别名
print(m.sqrt(25))      # 5.0
```

常用内置模块

```
import random # 随机数
import datetime # 日期时间
import json # JSON 处理
import re # 正则表达式
import os # 操作系统接口
import sys # 系统参数
import math # 数学函数
```

15. 文件读写

读文件

```
# 方式1: read() 全部读取
with open('file.txt', 'r', encoding='utf-8') as f:
    content = f.read()

# 方式2: 逐行读取
with open('file.txt', 'r', encoding='utf-8') as f:
    for line in f:
        print(line.strip())

# 方式3: readlines() 返回列表
with open('file.txt', 'r', encoding='utf-8') as f:
    lines = f.readlines()
```

写文件

```
# 覆盖写入
with open('file.txt', 'w', encoding='utf-8') as f:
    f.write("Hello, World!\n")

# 追加写入
with open('file.txt', 'a', encoding='utf-8') as f:
    f.write("追加一行\n")
```

文件模式

> 强烈推荐使用 with 语句，它会自动关闭文件，即使发生错误。

16. 错误与异常处理

```
try:
    num = int(input("请输入一个数字: "))
    print(10 / num)
except ValueError:
    print("请输入有效的数字！")
except ZeroDivisionError:
    print("不能除以零！")
except Exception as e:
    print(f"出了其他错误: {e}")
else:
    print("没有异常时执行") # try成功才执行
finally:
    print("无论如何都会执行") # 清理资源
```

常见异常类型

```
ValueError    # 值错误 (如 int("abc"))
TypeError     # 类型错误
ZeroDivisionError # 除以零
FileNotFoundError # 文件不存在
KeyError      # 字典键不存在
IndexError    # 列表索引越界
```

17. 面向对象编程

类和实例

```
class Person:
    """人类"""

    # 类变量（所有实例共享）
    species = "人"

    # 初始化方法（构造函数）
    def __init__(self, name, age):
        self.name = name    # 实例变量
        self.age = age

    # 实例方法
    def greet(self):
        return f"我叫{self.name}，今年{self.age}岁"

    # 修改方法
    def birthday(self):
        self.age += 1

# 创建实例（对象）
p1 = Person("小明", 18)
print(p1.greet()) # 我叫小明，今年18岁
p1.birthday()
print(p1.age)    # 19
```

继承

```
class Student(Person): # 继承 Person 类
    def __init__(self, name, age, grade):
        super().__init__(name, age) # 调用父类初始化
        self.grade = grade

    # 重写方法
    def greet(self):
        return f"我是{self.name}，{self.grade}年级学生"

s = Student("小红", 15, "九")
print(s.greet()) # 我是小红，九年级学生
```

18. 推荐学习路线

零基础入门路线

1. 基础语法：变量、数据类型、条件、循环 → 先会写简单脚本 2. 函数：def、参数、返回值 → 代码复用 3. 数据结构：list、dict → 处理数据 4. 文件操作：读写文件 → 实际项目必备 5. 模块：import → 利用现成的库 6. 面向对象：class → 构建复杂系统 7. 实战练习 → 做小项目

推荐在线资源

练习项目建议

1. 计算器 → 练习输入输出、基本运算 2. 猜数字游戏 → 练习循环、条件、随机数 3. 待办事项清单(Todo list) → 练习列表操作、文件读写 4. 密码生成器 → 练习字符串操作、random 5. 爬取网页数据 → 练习 requests + beautifulsoup

> 最后的话：Python

是最好入门的编程语言之一。不用死记硬背，边学边写代码，遇到错误是正常的——查错误信息、搜解决方案、调试，这才是真正的学习过程。加油！